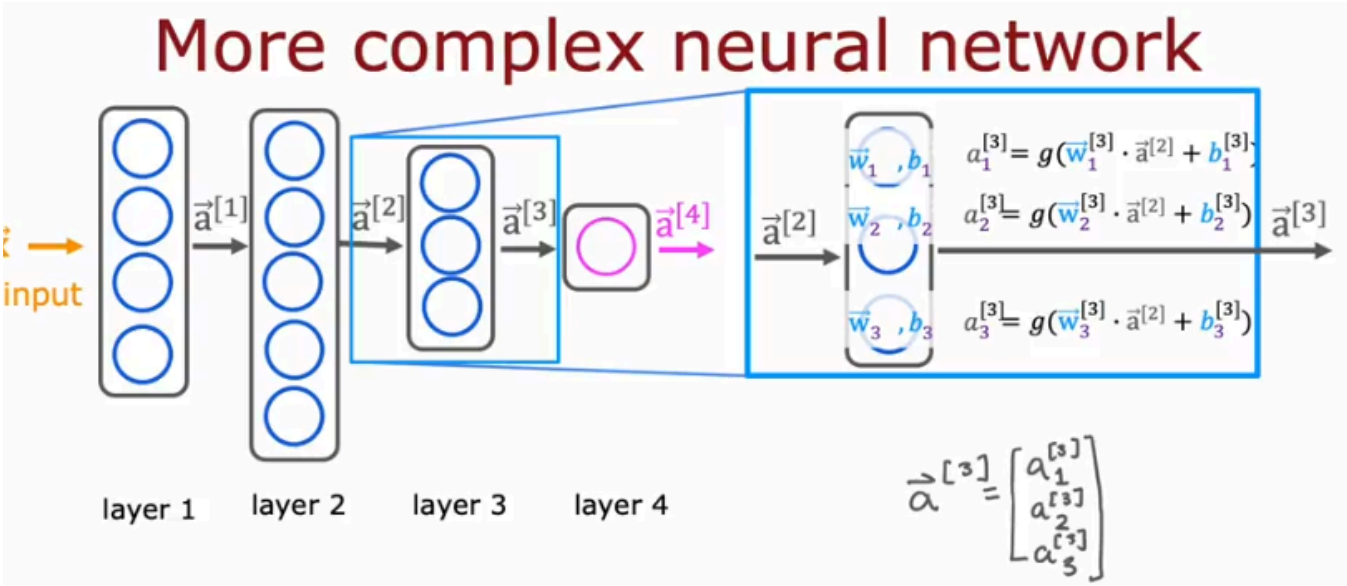
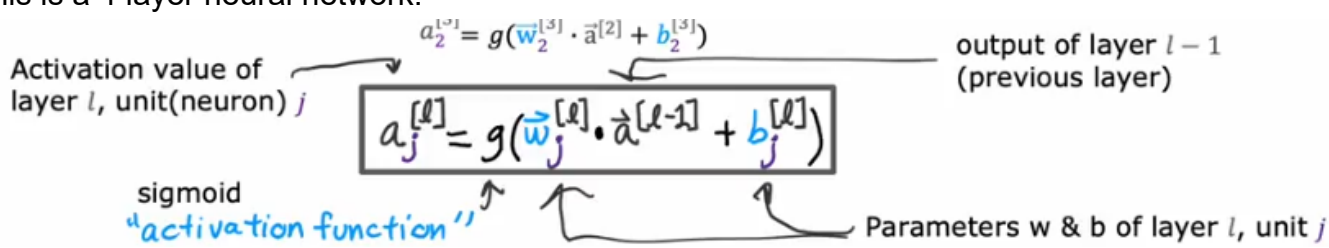
Day 13. More complex neural networks and TensorFlow

More complex neural networks

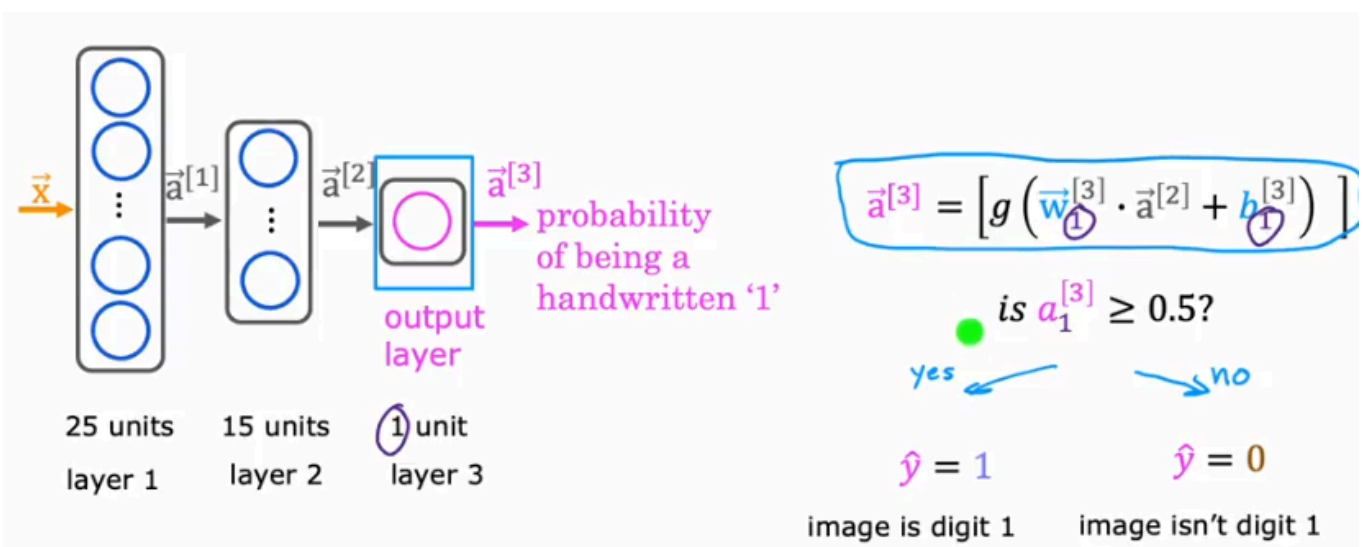


this is a 4 layer neural network.



Inference: making predictions (forward propagation)

Example: handwritten digit recognition



> At the end, it is still a function.

neurons lab

```
import tensorflow as tf
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras import Sequential
from tensorflow.keras.losses import MeanSquaredError, BinaryCrossentropy
from tensorflow.keras.activations import sigmoid
```

in linear regression:

```
linear_layer = Dense(units=1, activation = 'linear', )
a1 = linear_layer(X_train[0].reshape(1, 1))
w, b = linear_layer.get_weights()
linear_layer.set_weights([set_w, wet_b])
prediction = linear_layer(X_train)
```

So here, we can see that the `Dense` object is actually a layer that can be specified with activation function and number of units, and can be set weights and get weights, calling on the object will return the predicting result.

Neuron with Sigmoid activation

- logistic neuron

```
model = Sequential(
     [
         Dense(1, input_dim=1, activation = 'sigmoid', name = 'L1')
     ]
)
```

Question

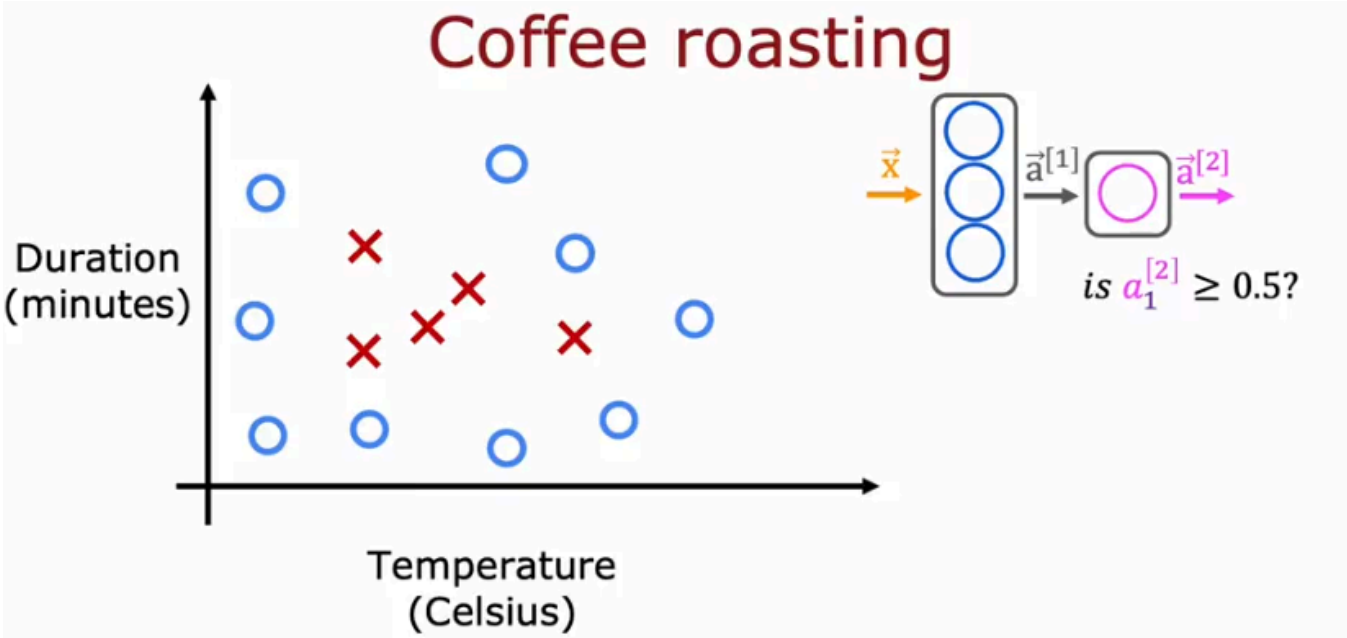
1. what exactly is `Dense`?
2. what is `Sequential`? Is there any other model?

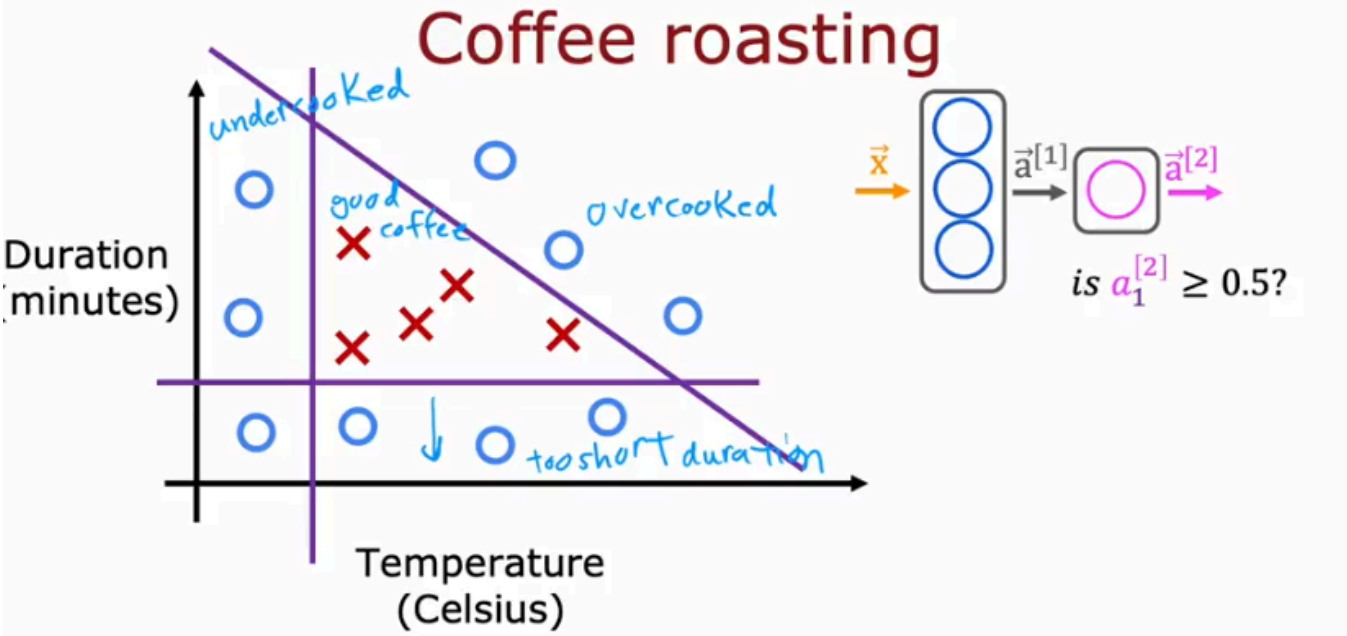
TensorFlow implementation

two leading frameworks to implement deep learning algorigthms: tensorflow and pytorch.

Inference in Code

Coffee roasting





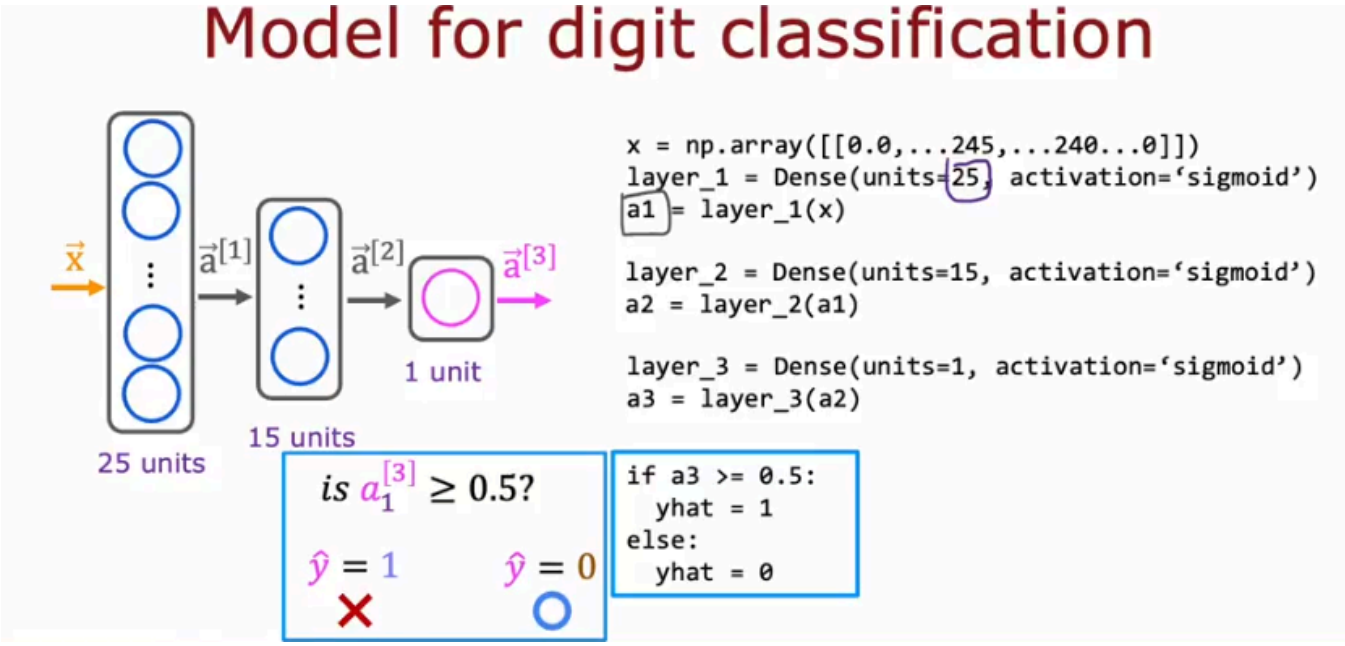
inference process

```
import tensorflow as tf
from tensorflow.keras.layers import Dense

x = np.array([[200.00, 17.0]])
layer_1 = Dense(units=3, activation='sigmoid')
a1 = layer_1(x)

layer_2 = Dense(units=1, activation='sigmoid')
a2 = layer_2(a1)
```

Model for digit classification



Data in TensorFlow

there are some differences in how the data are represented in Numpy and TensorFlow, and it is important to be aware of it.

Note about numpy arrays

3 columns

2 rows

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

2 x 3 matrix

```
x = np.array([[1, 2, 3],
              [4, 5, 6]])
```

```
[[1, 2, 3],
 [4, 5, 6]]
```

2D array

2 x 3

4 rows

$$\begin{bmatrix} 0.1 & 0.2 \\ -3 & -4 \\ -.5 & -.6 \\ 7 & 8 \end{bmatrix}$$

2 columns

4 x 2 matrix

```
x = np.array([[0.1, 0.2],
              [-3.0, -4.0,],
              [-0.5, -0.6,],
              [7.0, 8.0,]])
```

```
[[0.1, 0.2],
 [-3.0, -4.0,],
 [-0.5, -0.6,],
 [7.0, 8.0,]]
```

4 x 2

1 x 2

2 x 1

Note about numpy arrays

```
x = np.array([[200, 17]])
```

$$\begin{bmatrix} 200 & 17 \end{bmatrix}$$

1 x 2

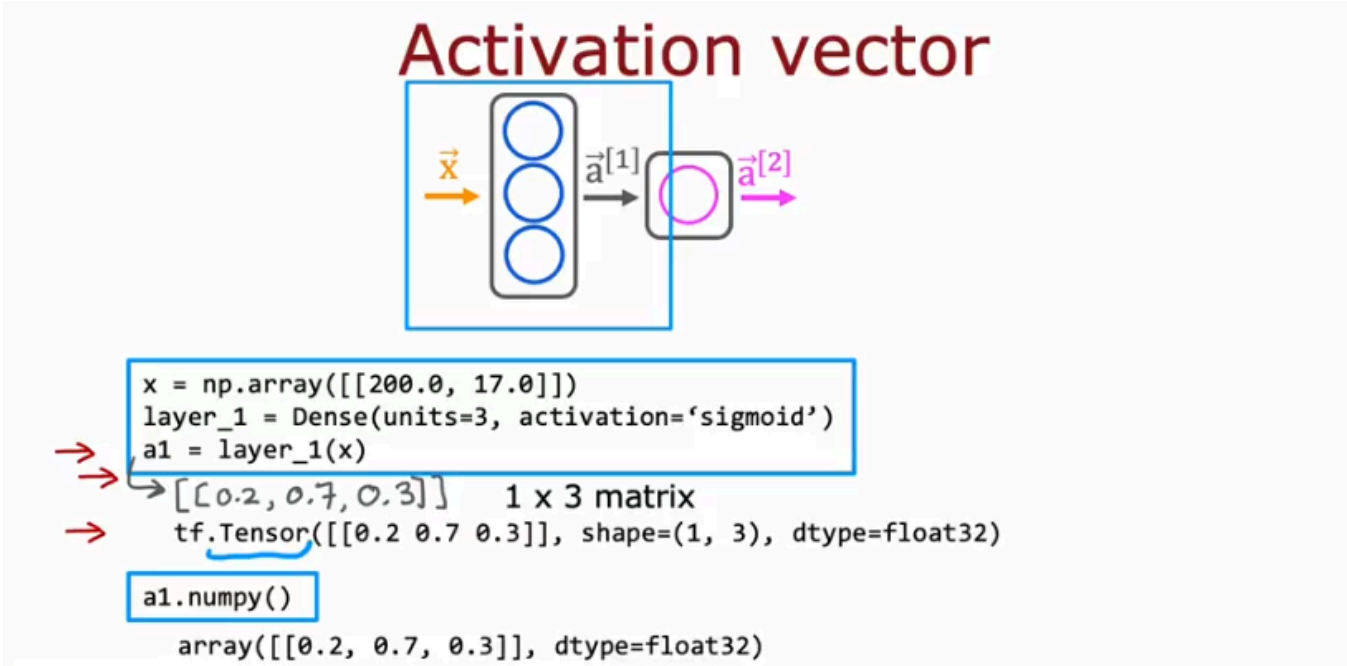
```
x = np.array([[200],
              [17]])
```

$$\begin{bmatrix} 200 \\ 17 \end{bmatrix}$$

2 x 1

```
x = np.array([200,17])
```

1D
"Vector"



we only need to be aware of that the original dataframe tf returns is not ndarray but tf.Tensor, so remember to turn it to numpy if you need ndarray (`a1.numpy()`).